# EXPERIMENT 13

## Case Study on Limitations of Low-Code Platforms for a Complex Enterprise Use Case

**Name:** Jobeson J | **Register Number:** 240701217
**Subject:** Low Code Application Development using Zoho Creator
**Project:** Bank Loan Application & Pre-Screening Management

## Title

Case study on the limitations of low-code platforms for a complex enterprise use case: an enterprise Loan Origination and Servicing System.

## Aim

To study where low-code platforms fall short when the Loan Application grows from a branch pre-screening tool into an enterprise system, and to identify when pro-code extensions or a hybrid architecture become necessary.

## Objectives

- To describe an enterprise loan system from applicant to analytics.
- To map the standard low-code capabilities (forms, reports, workflows, data model, roles) to its operations.
- To identify the limitations that appear at enterprise scale.
- To recommend a hybrid architecture where required.

## The Enterprise Use Case

The branch application of this lab series captures applications, pre-screens them, tracks stages and sends reminders. An enterprise bank would additionally run many branches, millions of records, a credit-scoring engine, integration with core banking and payment systems, regulatory reporting, and strict audit and availability requirements.

## What Low-Code Handles Well

**Table 13.1:** Operations well served by low-code

| Operation | Low-code capability used |
|---|---|
| Application intake | Forms with basic and advanced fields, validation, public link |
| Pre-screening and routing | Workflows, lookups, subforms |
| Stage control and approvals | Blueprints, approvals |
| Monitoring | Reports, pages, charts |
| Access control | Roles, profiles, data sharing |
| Reminders and notifications | Schedules, batch workflows, e-mail |

## Limitations at Enterprise Scale

**Table 13.2:** Limitations of low-code at enterprise scale (verify current limits against the platform documentation and plan)

| Area | Limitation | Why it matters for loans |
|---|---|---|
| Data volume and performance | Platform limits on records, API calls and processing time; large reports and heavy scripts slow down | Many branches and years of history |
| Complex calculations | Deluge is suited to short business rules; credit scoring, risk models and amortisation schedules are better in specialised code | Accuracy and testability of financial logic |
| Integration depth | Standard connectors cover common services; legacy core-banking systems often need custom APIs and middleware | Single view of the customer |
| Testing and version control | Limited automated testing and fine-grained source control compared with code repositories | Safe change in a regulated setting |
| Vendor lock-in | Applications, connections and AI models are tied to the platform | Migration cost and exit risk |
| Compliance and audit | Controls exist, but regulators may require specific audit trails, data residency and certifications | Regulated lending |
| Availability and plan limits | Features and limits depend on the subscription plan; trial accounts expire | Continuity of service |

## Hybrid Architecture

A hybrid approach keeps the user-facing and workflow parts in low-code and moves the heavy parts into pro-code services that the low-code application calls.

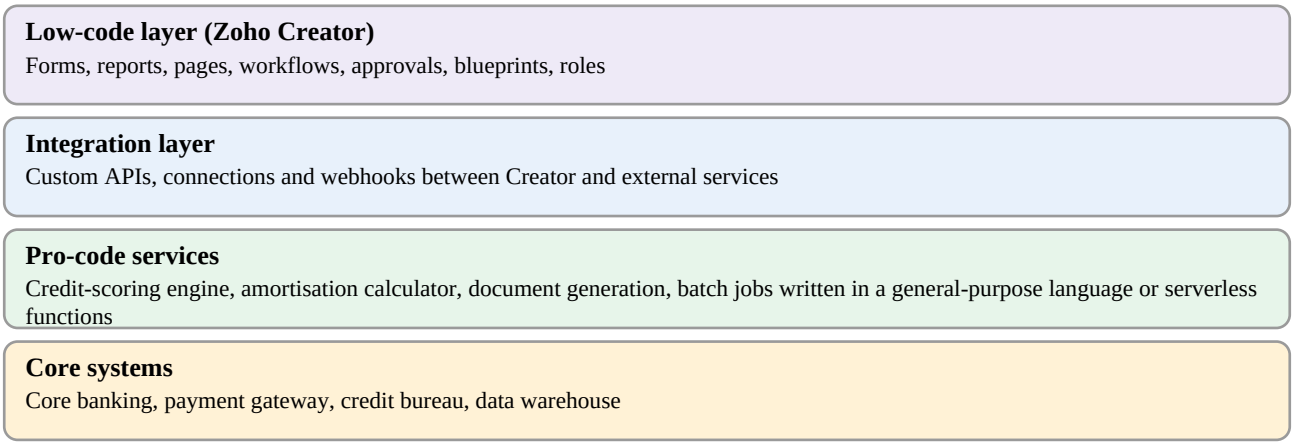



*Fig 13.1 — Hybrid architecture for the enterprise loan system (schematic; not a screenshot)*

## When to Move to Pro-Code

- The rule is long, mathematical or needs unit tests (for example reducing-balance schedules).
- The data volume or call rate approaches platform limits.
- A system outside Zoho must be called with a custom protocol.
- The regulator requires controls that the platform cannot provide.

## Recommendation

Keep intake, tracking, approvals and dashboards in Zoho Creator, because these give the speed and ease of change that low-code is good at. Move credit scoring, large-scale calculation and core-banking integration to pro-code services behind custom APIs, and keep both sides under the same governance (environments, roles and documentation).

## Output

**SCREENSHOT TO ATTACH**

*Optional: Microservices → Custom API page showing where a pro-code service would be registered*

*Fig 13.2 — Custom API area of Microservices*

## Result

The limitations of low-code for an enterprise loan system were identified and mapped to a hybrid architecture.

## Conclusion

Low-code is a strong choice for intake, workflow and visibility, but it is not the right place for heavy financial calculation, very large data volumes or deep legacy integration. Recognising the boundary early avoids rebuilding later.